

INTRODUÇÃO À ANÁLISE DE DADOS EM FÍSICA DE ALTAS ENERGIAS

Exercícios de ROOT

Professores: Dilson de Jesus Damião, Mauricio Thiel e Eliza Melo

Aluno: Thiago Henrique de Sousa

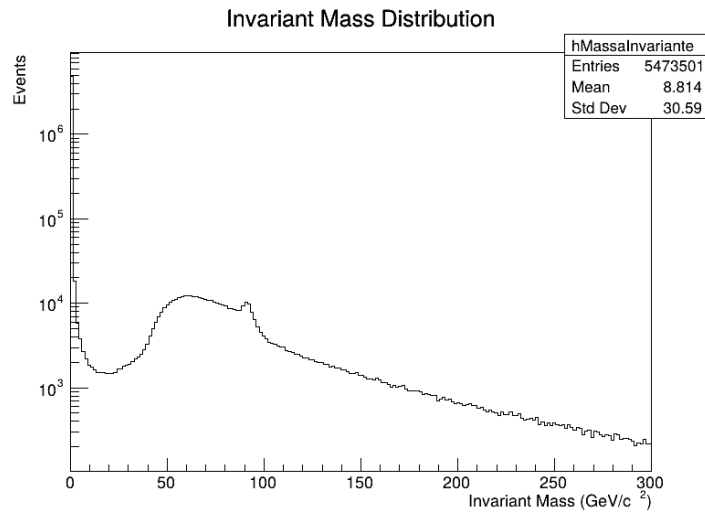
(12/11/2024)

EXERCÍCIO 1

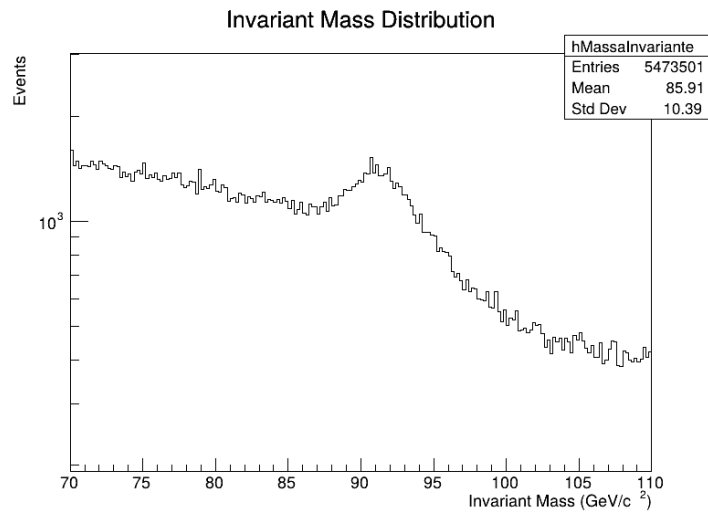
Foi feito um corte em p_T e η de: $p_T > 20\text{GeV}$ e $\eta < 2.5$ e calculamos a massa utilizando a fórmula:

$$M = \sqrt{2p_{T1}p_{T2}(\cosh(\eta_1 - \eta_2) - \cos(\phi_1 - \phi_2))}$$

Onde cada p_{T1} e p_{T2} é referente a um lépton selecionado e análogo para as pseudorapidez. Com a aplicação desses cortes, o número de eventos diminuiu, pois selecionamos uma região específica, eliminando as partículas de baixa energia e as partículas com valores mais altos de pseudorapidez. Esse corte representa a região de aceitação do CMS/LHC. Obtemos os seguintes gráficos:

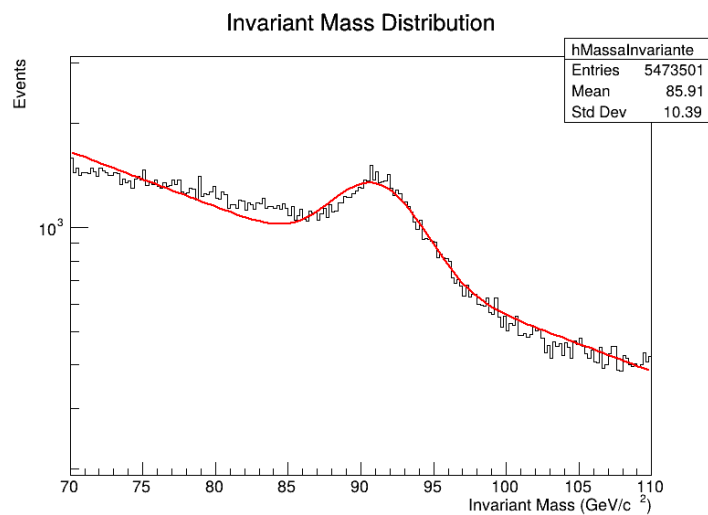


Fonte: O autor.



Fonte: O autor.

EXERCÍCIO 2 Fazer o ajuste da curva.



Fonte: O autor.

Códigos - ROOT Segue os códigos de cada tarefa respectivamente.

Código para os cortes.

```

1
2  #include <TTree.h>
3  #include <TTreeReader.h>
4  #include <TTreeReaderArray.h>
5  #include <TCanvas.h>
6  #include <TH1F.h>
7  #include <TMath.h>
8  #include <iostream>
9  #include <vector>
10 #include <filesystem>

```

```

11  #include <algorithm>
12  #include <string>
13
14  double calcular_massa_invariante(const std::vector<float>& pt, const std::vector<float>& eta, const std::vector<float>& phi) {
15      if (pt.size() >= 2) {
16          return sqrt(2 * pt[0] * pt[1] * (TMath::CosH(eta[0] - eta[1]) - TMath::Cos(phi[0] - phi[1])))
17      }
18      return -1.0;
19  }
20
21  void Z_mass_after_cuts() {
22      std::vector<std::string> diretorios = {
23          "/opendata/eos/opendata/cms/mc/RunIISummer20UL16NanoAODv9/ZZTo4L_TuneCP5_13TeV_powheg_pythia8",
24          "/opendata/eos/opendata/cms/mc/RunIISummer20UL16NanoAODv9/ZZTo4L_TuneCP5_13TeV_powheg_pythia8",
25      };
26
27
28      std::vector<double> e_massas_invariantes;
29
30      // Histogramas com faixa focada em torno da massa do Z (80 a 100 GeV)
31      TH1F* hMassaInvariante = new TH1F("hMassaInvariante", "Invariant Mass Distribution", 200, 70, 110);
32
33      for (const auto& diretorio : diretorios) {
34          for (const auto& entry : std::filesystem::directory_iterator(diretorio)) {
35              std::string file_path = entry.path();
36              TFile file(file_path.c_str(), "READ");
37              if (!file.IsOpen()) continue;
38
39              TTreeReader reader("Events", &file);
40              TTreeReaderArray<float> Electron_pt(reader, "Electron_pt");
41              TTreeReaderArray<float> Electron_eta(reader, "Electron_eta");
42              TTreeReaderArray<float> Electron_phi(reader, "Electron_phi");
43              TTreeReaderArray<float> Muon_pt(reader, "Muon_pt");
44              TTreeReaderArray<float> Muon_eta(reader, "Muon_eta");
45              TTreeReaderArray<float> Muon_phi(reader, "Muon_phi");
46              TTreeReaderArray<float> Jet_pt(reader, "Jet_pt");
47              TTreeReaderArray<float> Jet_eta(reader, "Jet_eta");
48              TTreeReaderArray<float> Jet_phi(reader, "Jet_phi");
49              TTreeReaderArray<float> Tau_pt(reader, "Tau_pt");
50              TTreeReaderArray<float> Tau_eta(reader, "Tau_eta");
51              TTreeReaderArray<float> Tau_phi(reader, "Tau_phi");
52
53              while (reader.Next()) {
54                  // Vetor de leptons com pT e eta
55                  std::vector<std::pair<float, int>> leptons; // (pT, indice)
56                  for (size_t i = 0; i < Electron_pt.GetSize(); ++i) {
57                      if (Electron_pt[i] > 20 && fabs(Electron_eta[i]) < 2.5) {
58                          leptons.emplace_back(Electron_pt[i], i); // Elétrons
59                      }
60                  }
61                  for (size_t i = 0; i < Muon_pt.GetSize(); ++i) {
62                      if (Muon_pt[i] > 20 && fabs(Muon_eta[i]) < 2.5) {
63                          leptons.emplace_back(Muon_pt[i], i + Electron_pt.GetSize()); // Muons
64                      }
65                  }
66                  for (size_t i = 0; i < Tau_pt.GetSize(); ++i) {
67                      if (Tau_pt[i] > 20 && fabs(Tau_eta[i]) < 2.5) {
68                          leptons.emplace_back(Tau_pt[i], i + Electron_pt.GetSize() + Muon_pt.GetSize());
69                      }
70                  }
71
72                  // Cálculo da massa invariante para cada par de leptons
73                  for (size_t i = 0; i < leptons.size(); ++i) {
74                      for (size_t j = i + 1; j < leptons.size(); ++j) {
75                          double massa = calcular_massa_invariante({leptons[i].first, leptons[j].first}, {leptons[i].second, leptons[j].second}, {leptons[i].second, leptons[j].second});
76                          if (massa > 0) {
77                              hMassaInvariante->Fill(massa);
78                          }
79                      }
80                  }
81              }
82          }
83      }
84
85      // Salva o histograma
86      hMassaInvariante->Write();
87  }

```

```

69     }
70 }
71
72 // Ordenar pT de forma decrescente
73 std::sort(leptons.rbegin(), leptons.rend());
74
75 // Seleciona os dois leptons com maior pT
76 if (leptons.size() >= 2) {
77     int idx1 = leptons[0].second;
78     int idx2 = leptons[1].second;
79
80     float pt1, eta1, phi1, pt2, eta2, phi2;
81
82     // Atribuir valores de pt, eta, phi de cada lepton
83     if (idx1 < Electron_pt.GetSize()) {
84         pt1 = Electron_pt[idx1];
85         eta1 = Electron_eta[idx1];
86         phi1 = Electron_phi[idx1];
87     } else if (idx1 < Electron_pt.GetSize() + Muon_pt.GetSize()) {
88         pt1 = Muon_pt[idx1 - Electron_pt.GetSize()];
89         eta1 = Muon_eta[idx1 - Electron_pt.GetSize()];
90         phi1 = Muon_phi[idx1 - Electron_pt.GetSize()];
91     } else {
92         pt1 = Tau_pt[idx1 - Electron_pt.GetSize() - Muon_pt.GetSize()];
93         eta1 = Tau_eta[idx1 - Electron_pt.GetSize() - Muon_pt.GetSize()];
94         phi1 = Tau_phi[idx1 - Electron_pt.GetSize() - Muon_pt.GetSize()];
95     }
96
97     if (idx2 < Electron_pt.GetSize()) {
98         pt2 = Electron_pt[idx2];
99         eta2 = Electron_eta[idx2];
100        phi2 = Electron_phi[idx2];
101    } else if (idx2 < Electron_pt.GetSize() + Muon_pt.GetSize()) {
102        pt2 = Muon_pt[idx2 - Electron_pt.GetSize()];
103        eta2 = Muon_eta[idx2 - Electron_pt.GetSize()];
104        phi2 = Muon_phi[idx2 - Electron_pt.GetSize()];
105    } else {
106        pt2 = Tau_pt[idx2 - Electron_pt.GetSize() - Muon_pt.GetSize()];
107        eta2 = Tau_eta[idx2 - Electron_pt.GetSize() - Muon_pt.GetSize()];
108        phi2 = Tau_phi[idx2 - Electron_pt.GetSize() - Muon_pt.GetSize()];
109    }
110
111    // Calcular massa invariante
112    std::vector<float> pt_values = {pt1, pt2};
113    std::vector<float> eta_values = {eta1, eta2};
114    std::vector<float> phi_values = {phi1, phi2};
115    double massa_invariante = calcular_massa_invariante(pt_values, eta_values, phi_v
116
117    if (massa_invariante >= 0) {
118        e_massas_invariantes.push_back(massa_invariante);
119    }
120 }
121 }
122 }
123 }
124
125
126 // Plot da Massa Invariante

```

```

127     TCanvas* canvas = new TCanvas("canvasInvariantMass", "Invariant Mass Distribution", 800, 600);
128     hMassaInvariante->SetLineColor(kBlack);
129     canvas->SetLogy();
130     for (const auto& massa : e_massas_invariantes) {
131         if (massa >= 0) hMassaInvariante->Fill(massa);
132     }
133     hMassaInvariante->Draw();
134     hMassaInvariante->GetXaxis()->SetTitle("Invariant Mass (GeV/c^{2})");
135     hMassaInvariante->GetYaxis()->SetTitle("Events");
136     canvas->SaveAs("invariant_mass_Z_v2.png");
137
138     delete canvas;
139     delete hMassaInvariante;
140 }

```

Código para o ajuste com os cortes aplicados.

```

1
2  #include <TTree.h>
3  #include <TTreeReader.h>
4  #include <TTreeReaderArray.h>
5  #include <TCanvas.h>
6  #include <TH1F.h>
7  #include <TMath.h>
8  #include <RooFit.h>
9  #include <RooRealVar.h>
10 #include <RooDataHist.h>
11 #include <RooCBShape.h>
12 #include <RooExponential.h>
13 #include <RooAddPdf.h>
14 #include <RooPlot.h>
15 #include <iostream>
16 #include <vector>
17 #include <filesystem>
18 #include <algorithm>
19 #include <string>
20
21 double calcular_massa_invariante(const std::vector<float>& pt, const std::vector<float>& eta, const
22     if (pt.size() >= 2) {
23         return sqrt(2 * pt[0] * pt[1] * (TMath::CosH(eta[0] - eta[1]) - TMath::Cos(phi[0] - phi[1])));
24     }
25     return -1.0;
26 }
27
28 void Z_mass_after_cuts_fit() {
29     // Diretórios para análise
30     std::vector<std::string> diretorios = {
31         "/opendata/eos/opendata/cms/mc/RunIISummer20UL16NanoAODv9/ZZTo4L_TuneCP5_13TeV_powheg_pythia8",
32         "/opendata/eos/opendata/cms/mc/RunIISummer20UL16NanoAODv9/ZZTo4L_TuneCP5_13TeV_powheg_pythia8",
33     };
34
35     std::vector<double> e_massas_invariantes;
36
37     for (const auto& diretorio : diretorios) {
38         for (const auto& entry : std::filesystem::directory_iterator(diretorio)) {
39             std::string file_path = entry.path();
40             TFile file(file_path.c_str(), "READ");

```

```

41     if (!file.IsOpen()) continue;
42
43     TTreeReader reader("Events", &file);
44     TTreeReaderArray<float> Electron_pt(reader, "Electron_pt");
45     TTreeReaderArray<float> Electron_eta(reader, "Electron_eta");
46     TTreeReaderArray<float> Electron_phi(reader, "Electron_phi");
47     TTreeReaderArray<float> Muon_pt(reader, "Muon_pt");
48     TTreeReaderArray<float> Muon_eta(reader, "Muon_eta");
49     TTreeReaderArray<float> Muon_phi(reader, "Muon_phi");
50     TTreeReaderArray<float> Tau_pt(reader, "Tau_pt");
51     TTreeReaderArray<float> Tau_eta(reader, "Tau_eta");
52     TTreeReaderArray<float> Tau_phi(reader, "Tau_phi");
53
54     while (reader.Next()) {
55         std::vector<std::pair<float, int>> leptons;
56         for (size_t i = 0; i < Electron_pt.GetSize(); ++i) {
57             if (Electron_pt[i] > 20 && fabs(Electron_eta[i]) < 2.5) {
58                 leptons.emplace_back(Electron_pt[i], i);
59             }
60         }
61         for (size_t i = 0; i < Muon_pt.GetSize(); ++i) {
62             if (Muon_pt[i] > 20 && fabs(Muon_eta[i]) < 2.5) {
63                 leptons.emplace_back(Muon_pt[i], i + Electron_pt.GetSize());
64             }
65         }
66         for (size_t i = 0; i < Tau_pt.GetSize(); ++i) {
67             if (Tau_pt[i] > 20 && fabs(Tau_eta[i]) < 2.5) {
68                 leptons.emplace_back(Tau_pt[i], i + Electron_pt.GetSize() + Muon_pt.GetSize());
69             }
70         }
71
72         std::sort(leptons.rbegin(), leptons.rend());
73
74         if (leptons.size() >= 2) {
75             int idx1 = leptons[0].second;
76             int idx2 = leptons[1].second;
77
78             float pt1, eta1, phi1, pt2, eta2, phi2;
79
80             if (idx1 < Electron_pt.GetSize()) {
81                 pt1 = Electron_pt[idx1];
82                 eta1 = Electron_eta[idx1];
83                 phi1 = Electron_phi[idx1];
84             } else if (idx1 < Electron_pt.GetSize() + Muon_pt.GetSize()) {
85                 pt1 = Muon_pt[idx1 - Electron_pt.GetSize()];
86                 eta1 = Muon_eta[idx1 - Electron_pt.GetSize()];
87                 phi1 = Muon_phi[idx1 - Electron_pt.GetSize()];
88             } else {
89                 pt1 = Tau_pt[idx1 - Electron_pt.GetSize() - Muon_pt.GetSize()];
90                 eta1 = Tau_eta[idx1 - Electron_pt.GetSize() - Muon_pt.GetSize()];
91                 phi1 = Tau_phi[idx1 - Electron_pt.GetSize() - Muon_pt.GetSize()];
92             }
93
94             if (idx2 < Electron_pt.GetSize()) {
95                 pt2 = Electron_pt[idx2];
96                 eta2 = Electron_eta[idx2];
97                 phi2 = Electron_phi[idx2];
98             } else if (idx2 < Electron_pt.GetSize() + Muon_pt.GetSize()) {

```

```

99         pt2 = Muon_pt[idx2 - Electron_pt.GetSize()];
100         eta2 = Muon_eta[idx2 - Electron_pt.GetSize()];
101         phi2 = Muon_phi[idx2 - Electron_pt.GetSize()];
102     } else {
103         pt2 = Tau_pt[idx2 - Electron_pt.GetSize() - Muon_pt.GetSize()];
104         eta2 = Tau_eta[idx2 - Electron_pt.GetSize() - Muon_pt.GetSize()];
105         phi2 = Tau_phi[idx2 - Electron_pt.GetSize() - Muon_pt.GetSize()];
106     }
107
108     std::vector<float> pt_values = {pt1, pt2};
109     std::vector<float> eta_values = {eta1, eta2};
110     std::vector<float> phi_values = {phi1, phi2};
111     double massa_invariante = calcular_massa_invariante(pt_values, eta_values, phi_v
112
113     if (massa_invariante >= 0) {
114         e_massas_invariantes.push_back(massa_invariante);
115     }
116 }
117 }
118 }
119 }
120
121 // 1. Criamos o histograma com os dados de massa invariante
122 TH1F* hMassaInvariante = new TH1F("hMassaInvariante", "Distribuição de Massa Invariante", 150, 6
123 for (const auto& massa : e_massas_invariantes) {
124     if (massa >= 0) hMassaInvariante->Fill(massa);
125 }
126
127 // 2. Definimos a variável para o RooFit
128 RooRealVar x("x", "Massa Invariante (GeV/c^{2})", 60, 120);
129
130 // 3. Criamos o RooDataHist a partir do histograma
131 RooDataHist data("data", "Dados de Massa Invariante", RooArgList(x), hMassaInvariante);
132
133 // 4. Definimos as funções para o sinal (Crystal Ball) e fundo (Exponencial)
134 RooRealVar mean("mean", "Média", 91.2, 90, 93); // Média do Z boson
135 RooRealVar sigma("sigma", "Desvio padrão", 2.3, 1.5, 3.0); // Largura do pico
136 RooRealVar alpha("alpha", "Parâmetro alpha", 1.3, 0.8, 2.5); // Parâmetro alpha (cauda)
137 RooRealVar n("n", "Parâmetro n", 7, 3, 15); // Parâmetro n (cauda)
138 RooCBShape signal("signal", "Função Crystal Ball", x, mean, sigma, alpha, n);
139
140 // Função de fundo exponencial
141 RooRealVar tau("tau", "Parâmetro Tau (Fundo Exponencial)", -0.2, -1.0, 0.0);
142 RooExponential background("background", "Fundo Exponencial", x, tau);
143
144 // 5. Criamos o modelo total como uma soma (signal + background)
145 RooRealVar fracSignal("fracSignal", "Fração de Sinal", 0.6, 0.2, 0.4); // Fração do sinal
146 RooAddPdf model("model", "Sinal + Fundo", RooArgList(signal, background), RooArgList(fracSignal)
147
148 // 6. Ajuste do modelo aos dados
149 model.fitTo(data, RooFit::PrintLevel(-1), RooFit::Range(60, 120));
150
151 // 7. Criamos o gráfico para o ajuste
152 RooPlot* frame = x.frame();
153 data.plotOn(frame);
154 model.plotOn(frame, RooFit::LineColor(kBlue)); // Ajuste total
155 model.plotOn(frame, RooFit::Components("signal"), RooFit::LineColor(kRed), RooFit::LineStyle(kDa
156 model.plotOn(frame, RooFit::Components("background"), RooFit::LineColor(kGreen), RooFit::LineSty

```

```

157
158 // 8. Criação do canvas e exibição do gráfico
159 TCanvas* canvas = new TCanvas("canvasFit", "Ajuste do Sinal + Fundo", 800, 600);
160 frame->SetTitle("Ajuste de Massa Invariante com Sinal + Fundo");
161 frame->GetXaxis()->SetTitle("Massa Invariante (GeV/c^{2})");
162 frame->GetYaxis()->SetTitle("Eventos");
163 frame->Draw();
164
165 // 9. Adiciona a legenda
166 TLegend* legend = new TLegend(0.7, 0.6, 0.9, 0.9);
167 legend->AddEntry(frame->getObject(0), "Dados", "P");
168 legend->AddEntry(frame->getObject(1), "Ajuste Total", "L");
169 legend->AddEntry(frame->getObject(2), "Sinal (Crystal Ball)", "L");
170 legend->AddEntry(frame->getObject(3), "Fundo (Exponencial)", "L");
171 legend->Draw();
172
173 // 10. Salva a imagem
174 canvas->SaveAs("mass_Z_fit.png");
175
176 // Libera a memória
177 delete canvas;
178 delete frame;
179 delete legend;
180 delete hMassaInvariante;
181 }

```
